**UNIVERSIDAD IBEROAMERICANA LEÓN**
Departamento de Ingenierías · Desarrollo de Aplicaciones y Servicios Virtuales
Docente titular: Ing. Alejandro Montes Moreno

# PRÁCTICA: PERSISTENCIA DE DATOS
# PERSISTENCIA Y DISEÑO MÍNIMO DE BASE DE DATOS

*COTIZA-ELEV — Sistema Web de Cotizaciones para Elevadores (ELEVESE)*

| Concepto institucional | Detalle de la entrega |
|---|---|
| Proyecto | COTIZA-ELEV — Sistema Web Centralizado de Cotizaciones y Solicitudes. |
| Organización cliente | ELEVESE (Instalación, modernización y mantenimiento de elevadores). |
| Asignatura | Desarrollo de Aplicaciones y Servicios Virtuales. |
| Docente titular | Ing. Alejandro Montes Moreno. |
| Equipo de desarrollo | Orlando Villalobos Gutiérrez y Diego Emilio Alférez Vallejo. |
| Asignación de roles | • Revisor técnico / Diseñador de datos: Orlando Villalobos Gutiérrez.<br>• Tester / Documentador: Diego Emilio Alférez Vallejo. |
| Decisión de persistencia | Ruta B: Base de datos relacional (MySQL 8.0 / InnoDB) vía PHP PDO. |
| Alcance del diseño | 9 tablas relacionales en 3FN, transacciones ACID, integridad referencial y aislamiento RBAC. |
| Fecha de entrega | Octubre de 2026. |

León, Guanajuato, México · 2026

## 1. Propósito de la práctica

El propósito fundamental de esta práctica consiste en analizar, justificar y definir formalmente la estrategia de persistencia de datos para la plataforma web COTIZA-ELEV de la empresa ELEVESE, seleccionando el paradigma de almacenamiento idóneo y proponiendo un diseño mínimo de base de datos riguroso antes de codificar los módulos del sistema. Esta planeación previene problemas graves de inconsistencia financiera, pérdida de información y cuellos de botella por concurrencia multiusuario.

En el contexto operativo de ELEVESE, la persistencia no puede resolverse con soluciones volátiles o archivos planos aislados. Las cotizaciones comerciales requieren un ciclo de vida duradero (Borrador, Emitida, Aceptada, Rechazada y Vencida), los activos mecánicos instalados (elevadores de pasajeros, montacargas y montaautos) tienen una vida útil de décadas que exige un historial de mantenimiento inmutable, y los técnicos en campo levantan solicitudes de refacciones que deben vincularse en tiempo real con las órdenes emitidas desde la oficina.

## 2. Objetivo de aprendizaje

Decidir con fundamentación técnica y metodológica cómo se gestionará la persistencia de datos en COTIZA-ELEV, verificando que la estructura generada esté alineada con los 43 requerimientos funcionales y 29 no funcionales del sistema, aplicando principios de normalización relacional (3FN), integridad referencial estricta, propiedades ACID para operaciones comerciales multi-partida y aislamiento estricto de seguridad para que el perfil técnico no tenga acceso a precios bajo ninguna circunstancia.

## 3. Entregable general y definición de roles

De acuerdo con la guía oficial de la práctica, este documento integra los cinco entregables exigidos:

- **1. Decisión sobre persistencia de datos:** Determinación formal de la ruta de almacenamiento (Ruta B: Base de datos relacional).
- **2. Justificación técnica de la decisión:** Análisis de concurrencia, transaccionalidad ACID y seguridad RBAC en servidor.
- **3. Diseño mínimo de base de datos:** Estructura completa de 9 tablas relacionales con sus atributos, llaves primarias y foráneas.
- **4. Módulos responsables de la persistencia:** Delimitación entre la infraestructura central (Database PDO) y repositorios por módulo.
- **5. Bitácora de uso de IA:** Evidencia crítica de las seis fases del trabajo asistido por IA, registrando aportes y correcciones.

### Asignación de roles y responsabilidades del equipo

| Integrante | Rol asignado | Responsabilidades y aportación técnica |
|---|---|---|
| **Orlando Villalobos Gutiérrez** | Revisor técnico / Diseñador de datos | Decidió formalmente la adopción de la Ruta B (MySQL/InnoDB). Diseñó la arquitectura de persistencia, el modelo relacional normalizado de 9 entidades, la estrategia transaccional ACID para partidas multi-concepto, la clase singleton PDO en PHP y la regla de no-borrado físico (baja lógica). Formuló los prompts de diseño y supervisó la coherencia técnica del esquema. |
| **Diego Emilio Alférez Vallejo** | Tester / Documentador | Realizó la validación cruzada entre los 43 requerimientos funcionales y las entidades propuestas. Diseñó los casos de prueba de integridad referencial, verificó que las consultas del perfil técnico omitieran campos de costos y tarifas (cumpliendo RF-05 y RNF-12), documentó el diccionario de datos de 28 campos y registró los desvíos y alucinaciones de la IA en la bitácora. |

## 4. Paso 1: Decidir la persistencia de datos

Antes de codificar el primer módulo del proyecto, el equipo respondió de forma sistemática a las siete preguntas guía de la actividad oficial para fundamentar la persistencia del sistema:

| Pregunta de la práctica | Respuesta fundamentada del equipo COTIZA-ELEV |
|---|---|
| ¿El proyecto requiere guardar información después de cerrar el programa? | Sí, obligatoriamente. El proceso de cotización de elevadores no es efímero; involucra ciclos comerciales que se extienden por semanas o meses, historiales de fallas mecánicas de equipos que operan durante más de 20 años y catálogos de piezas con vigencias legales de precios. Los datos deben preservarse permanentemente. |
| ¿Qué información se guardará? | 1) Cuentas de usuario y credenciales seguras (RBAC).<br>2) Clientes, razones sociales, RFC y contactos.<br>3) Activos mecánicos (elevadores, números de serie, paradas, capacidad de carga).<br>4) Catálogo maestro de refacciones, tarifas horarias y vigencias.<br>5) Solicitudes de servicio y levantamiento de refacciones por técnicos en campo.<br>6) Cotizaciones con partidas calculadas, subtotales, IVA y folios anuales.<br>7) Historial de emisión de propuestas PDF y bitácora de auditoría inmutable. |
| ¿Se usará base de datos? | Sí. Es el único mecanismo capaz de garantizar atomicidad transaccional, integridad referencial entre 9 módulos y concurrencia multiusuario entre técnicos móviles y personal administrativo en oficina. |
| ¿Qué alternativa se usará si no hay base de datos? | No aplica alternativa basada en archivos planos para el núcleo del sistema. Si se forzara a prescindir de BD, se tendrían que usar archivos JSON/CSV, pero estos provocarían condiciones de carrera (race conditions), corrupción de datos por escrituras concurrentes e incapacidad de bloquear el acceso a tarifas por el rol técnico. |
| ¿Qué tipo de base de datos se usará si aplica? | Base de datos relacional (RDBMS) MySQL 8.0 / MariaDB con motor de almacenamiento transaccional InnoDB, gestionada desde PHP mediante la extensión PDO con sentencias preparadas y soporte de transacciones ACID. |
| ¿Qué módulo administrará los datos? | Infraestructura central orquestada por la clase 'Database' (PDO Singleton en app/Core/Database.php), mientras que cada módulo funcional (M01 a M09) implementa repositorios dedicados que encapsulan las consultas SQL de su dominio. |
| ¿Qué requerimientos justifican esta decisión? | Los 43 requerimientos funcionales (RF-01 a RF-43) y especialmente RF-01 a RF-07 (cuentas y roles), RF-08 a RF-12 (directorio de clientes), RF-13 a RF-15 (ficha de activos), RF-16 a RF-19 (tarifas con vigencia), RF-20 a RF-24 (solicitudes técnicas), RF-25 a RF-34 (cotizaciones multi-partida), RF-35 a RF-38 (seguimiento y folios), RF-41 a RF-43 (auditoría), y los no funcionales RNF-12 (seguridad de precios), RNF-15 (integridad) y RNF-18 (retención 5 años). |

## 5. Paso 2: Elección de ruta de trabajo (Ruta B)

El equipo seleccionó formalmente la RUTA B: Proyectos que SÍ requieren base de datos. A continuación se expone la justificación técnica y de ingeniería de software que descarta el uso de archivos planos y valida la arquitectura elegida:

### Justificación técnica de la adopción del RDBMS MySQL / InnoDB

- **Concurrencia multiusuario real:** El modelo operativo de ELEVESE involucra técnicos en campo diagnosticando elevadores desde dispositivos móviles mientras simultáneamente dos administradores en oficina capturan partidas y emiten cotizaciones. El motor InnoDB resuelve bloqueos a nivel de fila (row-level locking), impidiendo que dos operaciones concurrentes sobre distintas cotizaciones se bloqueen o sobrescriban mutuamente.
- **Propiedades ACID en cotizaciones multi-partida:** Una cotización de elevador puede incorporar de 5 a 20 partidas (motores, guías, botoneras, cables de tracción, mano de obra y pólizas de mantenimiento). Si la inserción de la partida número 10 falla por validación de catálogo, la transacción completa debe revertirse ('ROLLBACK') para evitar cotizaciones incompletas o montos financieros desfasados. Esto es imposible de asegurar de forma atómica en archivos de texto.

- **Aislamiento estricto de seguridad RBAC (RF-05 y RNF-12):** La regla de negocio más crítica de ELEVESE prohíbe que el rol técnico conozca precios unitarios, márgenes de ganancia o tarifas comerciales. Con un RDBMS y sentencias preparadas en PDO, el backend asegura que las consultas ejecutadas por la vista del técnico jamás incluyan columnas financieras de las tablas 'catalogo_conceptos', 'cotizaciones' o 'partidas_cotizacion'. En un esquema basado en archivos (JSON/CSV), el archivo contendría los precios y quedaría expuesto a lectura indebida.
- **Integridad referencial y prevención de registros huérfanos:** Mediante llaves foráneas con restricciones 'ON DELETE RESTRICT', el motor de base de datos impide eliminar clientes que tienen cotizaciones emitidas, o borrar conceptos de catálogo que forman parte de una propuesta aceptada, preservando la validez legal y contable de los contratos.
- **Estandarización y compatibilidad de despliegue:** MySQL 8.0 / MariaDB es el estándar absoluto en infraestructura LAMP/cPanel, con soporte nativo para copias de seguridad consistentes (mysqldump), optimización de índices B-Tree y bajo consumo de memoria adecuado para las 8 semanas de desarrollo del curso.

## 6. Diseño mínimo de base de datos

El sistema COTIZA-ELEV se compone de nueve entidades relacionales normalizadas en Tercera Forma Normal (3FN), diseñadas para desacoplar el ciclo de acceso, el catálogo técnico, la operación de solicitudes y el motor comercial:

| Entidad / Tabla | Atributos principales | Llave primaria | Relaciones e integridad | RF vinculados |
|---|---|---|---|---|
| **usuarios [M01]** | nombre_completo, correo_electronico, password_hash, rol ('admin_ventas', 'tecnico'), intentos_fallidos, bloqueado_hasta, estatus ('activo', 'inactivo'), created_at | id_usuario (INT AUTO_INC) | 1 a N con solicitudes (id_tecnico); 1 a N con cotizaciones (id_usuario_creador); 1 a N con bitacora_auditoria (id_usuario). | RF-01, RF-02, RF-03, RF-04, RF-05, RF-06, RF-07 |
| **clientes [M02]** | clave_cliente, razon_social, rfc, nombre_contacto, correo, telefono, direccion_fiscal, estatus ('activo', 'inactivo'), created_at | id_cliente (INT AUTO_INC) | 1 a N con equipos_activos (id_cliente); 1 a N con cotizaciones (id_cliente); 1 a N con solicitudes (id_cliente). | RF-08, RF-09, RF-10, RF-11, RF-12 |
| **equipos_activos [M03]** | numero_serie, tipo_equipo ('pasajeros', 'carga', 'montaautos'), marca, modelo, capacidad_kg, numero_paradas, ubicacion_instalacion, estatus, created_at | id_activo (INT AUTO_INC) | N a 1 con clientes (id_cliente); 1 a N con solicitudes (id_activo); 1 a N con cotizaciones (id_activo). | RF-13, RF-14, RF-15 |
| **catalogo_conceptos [M04]** | codigo_sku, tipo_concepto ('refaccion', 'mano_obra', 'poliza', 'modernizacion'), descripcion, unidad_medida, precio_base, vigente_desde, vigente_hasta, estatus | id_concepto (INT AUTO_INC) | 1 a N con partidas_cotizacion (id_concepto). | RF-16, RF-17, RF-18, RF-19 |
| **solicitudes [M05]** | folio_solicitud, tipo_servicio ('preventivo', 'correctivo', 'modernizacion'), diagnostico_falla, piezas_requeridas, estado ('pendiente', 'en_revision', 'cotizada', 'rechazada') | id_solicitud (INT AUTO_INC) | N a 1 con usuarios (id_tecnico); N a 1 con clientes (id_cliente); N a 1 con equipos_activos (id_activo); 1 a 0..1 con cotizaciones (id_solicitud). | RF-20, RF-21, RF-22, RF-23, RF-24 |
| **cotizaciones [M06 / M07]** | folio_cotizacion, tipo_cotizacion, estado ('borrador', 'emitida', 'aceptada', 'rechazada', 'vencida'), fecha_emision, fecha_vencimiento, subtotal, porcentaje_descuento, monto_descuento, iva, total, moneda | id_cotizacion (INT AUTO_INC) | N a 1 con clientes (id_cliente); N a 1 con equipos_activos (id_activo); N a 1 con usuarios (id_usuario_creador); N a 1 opcional con solicitudes; 1 a N con partidas_cotizacion; 1 a N con seguimiento_propuestas. | RF-25, RF-26, RF-31, RF-32, RF-33, RF-34 |
| **partidas_cotizacion [M06 / M07]** | numero_item, descripcion_concepto, cantidad, precio_unitario, subtotal_partida, descuento_partida, total_partida, observaciones | id_partida (INT AUTO_INC) | N a 1 con cotizaciones (id_cotizacion); N a 1 con catalogo_conceptos (id_concepto). | RF-27, RF-28, RF-29, RF-30 |
| **seguimiento_propuestas [M08]** | folio_propuesta, version_pdf, ruta_archivo_pdf, fecha_envio_cliente, medio_envio, resultado ('en_espera', 'ganada', 'perdida', 'renegociacion'), motivo_rechazo, fecha_cierre | id_seguimiento (INT AUTO_INC) | N a 1 con cotizaciones (id_cotizacion); N a 1 con usuarios (registrado_por). | RF-35, RF-36, RF-37, RF-38 |
| **bitacora_auditoria [M09]** | modulo_afectado, accion ('LOGIN', 'INSERT', 'UPDATE', 'DELETE', 'STATUS_CHANGE', 'SECURITY_ALERT'), ip_origen, user_agent, detalle_evento, payload_previo, payload_nuevo, created_at | id_log (BIGINT AUTO_INC) | N a 1 con usuarios (id_usuario, nullable para eventos de autenticación). | RF-41, RF-42, RF-43 |

## 7. Diccionario de datos básico y exhaustivo

A continuación se detalla la especificación de 28 campos representativos que cubren las llaves primarias, atributos técnicos, magnitudes físicas, cálculos monetarios y marcas de tiempo del sistema:

| Campo | Tipo de dato sugerido | Descripción funcional | Obligatorio | Ejemplo de dato |
|---|---|---|---|---|
| **id_cotizacion** | INT UNSIGNED AUTO_INCREMENT | Identificador único secuencial de la cotización. | Sí | 1042 |
| **folio_cotizacion** | VARCHAR(20) UNIQUE | Código correlativo comercial anual oficial con formato COT-AAAA-NNNN. | Sí | COT-2026-0042 |
| **id_cliente** | INT UNSIGNED | Llave foránea que referencia al cliente al que se cotiza. | Sí | 15 |
| **id_activo** | INT UNSIGNED | Llave foránea que referencia al elevador objeto del servicio. | Sí | 8 |
| **id_usuario_creador** | INT UNSIGNED | Llave foránea del administrador de ventas que generó la cotización. | Sí | 2 |
| **id_solicitud** | INT UNSIGNED NULL | Llave foránea opcional a la solicitud técnica que originó la cotización. | No | 107 |
| **subtotal** | DECIMAL(12,2) | Sumatoria de los importes de todas las partidas antes de descuento e IVA. | Sí | 148500.00 |
| **porcentaje_descuento** | DECIMAL(5,2) | Porcentaje de descuento comercial aplicado sobre el subtotal (0.00 a 100.00). | Sí | 5.00 |
| **monto_descuento** | DECIMAL(12,2) | Monto en pesos resultante del porcentaje de descuento comercial. | Sí | 7425.00 |
| **iva** | DECIMAL(12,2) | Impuesto al Valor Agregado (16%) calculado sobre la base neta. | Sí | 22572.00 |
| **total** | DECIMAL(12,2) | Monto final liquidable de la cotización en pesos mexicanos (MXN). | Sí | 163647.00 |
| **estado** | ENUM('borrador', 'emitida', 'aceptada', 'rechazada', 'vencida') | Estado comercial dentro del ciclo de vida de la cotización. | Sí | emitida |
| **fecha_emision** | DATE | Fecha en que se formaliza y emite la cotización al cliente. | Sí | 2026-10-15 |
| **fecha_vencimiento** | DATE | Fecha límite de vigencia comercial de los precios estipulados. | Sí | 2026-11-14 |
| **id_partida** | INT UNSIGNED AUTO_INCREMENT | Identificador único de cada partida individual cotizada. | Sí | 5320 |
| **cantidad** | DECIMAL(8,2) | Número de unidades, horas de trabajo o metros lineales cotizados. | Sí | 2.00 |
| **precio_unitario** | DECIMAL(12,2) | Precio unitario congelado al momento de emitir la cotización. | Sí | 35000.00 |
| **total_partida** | DECIMAL(12,2) | Importe total de la partida calculada (cantidad * precio_unitario). | Sí | 70000.00 |
| **correo_electronico** | VARCHAR(150) UNIQUE | Correo corporativo único del usuario para autenticación. | Sí | orlando@elevelse.mx |
| **password_hash** | VARCHAR(255) | Hash unidireccional de la contraseña generado con algoritmo bcrypt. | Sí | $2y$12$e8x... |
| **rol** | ENUM('admin_ventas', 'tecnico') | Rol asignado para control de acceso RBAC. | Sí | admin_ventas |
| **intentos_fallidos** | TINYINT UNSIGNED | Contador para bloqueo de fuerza bruta al alcanzar 5 intentos. | Sí | 0 |
| **bloqueado_hasta** | DATETIME NULL | Marca de tiempo de expiración del bloqueo temporal de 15 minutos. | No | 2026-10-15 14:30:00 |
| **rfc** | VARCHAR(13) UNIQUE | Registro Federal de Contribuyentes para facturación fiscal. | Sí | ELE120304AB5 |
| **razon_social** | VARCHAR(180) | Denominación legal completa de la empresa o cliente particular. | Sí | Torre Médica León S.A. |
| **numero_serie** | VARCHAR(50) UNIQUE | Número de placa o número de serie de fábrica del elevador. | Sí | SN-OTIS-2018-994 |
| **capacidad_kg** | INT UNSIGNED | Carga nominal máxima del elevador en kilogramos (100 a 5000 kg). | Sí | 1000 |
| **numero_paradas** | TINYINT UNSIGNED | Cantidad de pisos, niveles o niveles atendidos por el equipo (2 a 40). | Sí | 12 |

## 8. Operaciones de datos (CRUD)

Cada entidad del modelo participa en operaciones específicas según el rol del usuario conectado y las reglas de integridad del dominio comercial:

| Operación de datos | Descripción detallada y reglas de negocio | Módulos responsables |
|---|---|---|
| **Crear (INSERT)** | • Alta de usuarios con password_hash bcrypt y asignación de rol (M01).<br>• Registro de clientes con validación de RFC y teléfono (M02).<br>• Registro de elevadores activos asociados a un cliente existente (M03).<br>• Carga masiva o individual de conceptos del catálogo con vigencia (M04).<br>• Levantamiento de solicitud de piezas o falla mecánica por el técnico en campo (M05).<br>• Creación atómica de cotización con múltiples partidas dentro de una transacción PDO (M06/M07).<br>• Generación de registro de propuesta PDF con folio correlativo anual (M08).<br>• Registro inmutable de eventos de auditoría y excepciones (M09). | M01, M02, M03, M04, M05, M06, M07, M08, M09 |
| **Consultar (SELECT)** | • Validación de credenciales de acceso y comprobación de bloqueo temporal (M01).<br>• Búsqueda predictiva de clientes por RFC, razón social o clave (M02).<br>• Ficha técnica del elevador con validación de rangos mecánicos (M03).<br>• Consulta de tarifas vigentes al momento de cotizar (M04).<br>• Bandeja de solicitudes: el técnico consulta sus solicitudes sin montos; el admin consulta todas (M05).<br>• Consulta de cotizaciones por folio, cliente, fecha y estado comercial (M06).<br>• Extracción de partidas para el cálculo de totales en el motor matemático (M07).<br>• Consulta de datos para renderizado de la propuesta económica PDF (M08).<br>• Métricas ejecutivas y reportes exportables a formato CSV (M09). | M01, M02, M03, M04, M05, M06, M07, M08, M09 |
| **Actualizar (UPDATE)** | • Incremento de intentos fallidos y reseteo al iniciar sesión exitosamente (M01).<br>• Edición de datos de contacto o domicilio fiscal del cliente (M02).<br>• Actualización de ubicación o estatus de mantenimiento del activo (M03).<br>• Modificación de tarifas del catálogo o cierre de vigencia temporal (M04).<br>• Transición de estado de la solicitud técnica: Pendiente → Cotizada → Atendida (M05).<br>• Modificación de partidas en cotizaciones con estatus Borrador (M06/M07).<br>• Cambio de estado de la cotización: Borrador → Emitida → Aceptada / Rechazada (M06).<br>• Registro del resultado comercial de la propuesta (Ganada / Perdida) (M08). | M01, M02, M03, M04, M05, M06, M07, M08 |
| **Eliminar (Baja Lógica / Soft Delete)** | • POLÍTICA OBLIGATORIA DE NO BORRADO FÍSICO (DELETE CASCADE prohibido).<br>• La desactivación de usuarios se gestiona mediante bandera estatus = 'inactivo' (M01).<br>• Los clientes y elevadores con cotizaciones históricas no se eliminan físicamente; se marcan como inactivos para preservar la trazabilidad fiscal y técnica (M02, M03).<br>• Los conceptos obsoletos del catálogo se inactivan o se cierra su campo vigente_hasta (M04).<br>• Las cotizaciones emitidas jamás se eliminan; los borradores descartados se marcan como Cancelados (M06).<br>• La bitácora de auditoría es estrictamente inmutable (solo INSERT y SELECT) (M09). | M01, M02, M03, M04, M06, M09 |

## 9. Diagrama Entidad-Relación (ERD) visual

La Figura 1 ilustra el diagrama Entidad-Relación formal del sistema COTIZA-ELEV. En él se aprecian las nueve entidades estructuradas en tres columnas funcionales (Seguridad y Solicitudes, Catálogos Maestros, y Motor Comercial de Cotización), evidenciando las llaves primarias (PK), llaves foráneas (FK), atributos principales y las cardinalidades que rigen el intercambio de información:

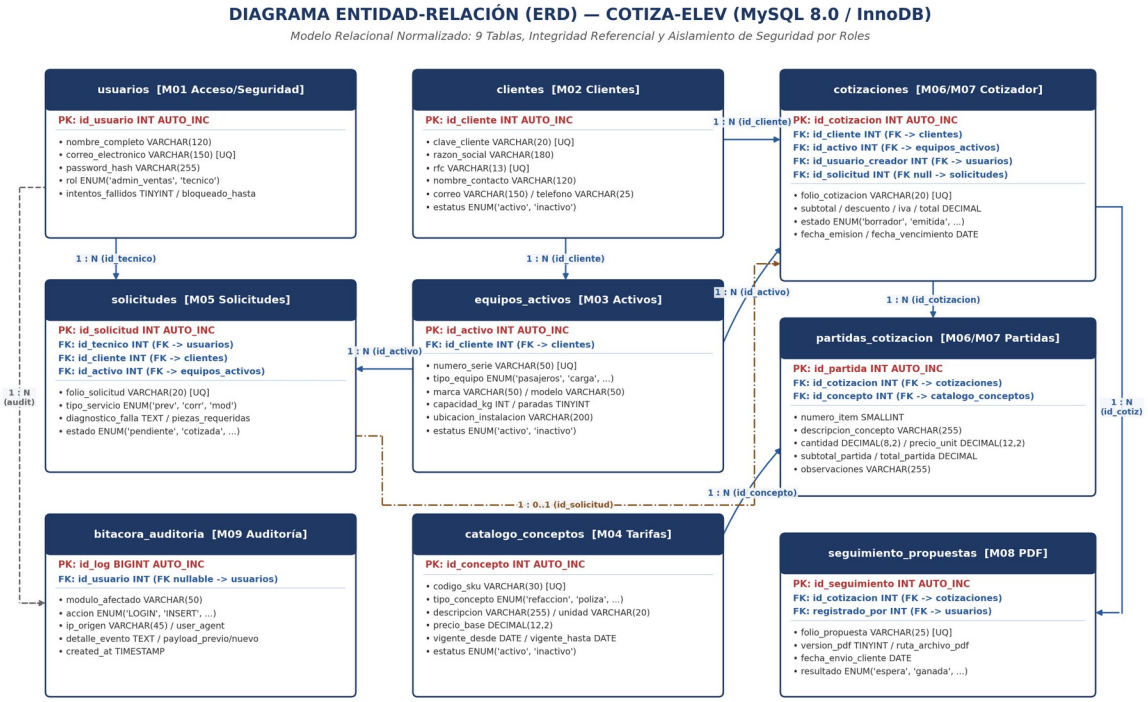



*Figura 1. Diagrama Entidad-Relación (ERD) del sistema COTIZA-ELEV con 9 tablas relacionales en MySQL 8.0 / InnoDB.*

### Descripción técnica de cardinalidades y flujos de datos del ERD

- **usuarios (1) ──< N solicitudes (id_tecnico):** Un técnico levanta múltiples solicitudes en campo; cada una queda vinculada al técnico que inspeccionó el equipo.
- **clientes (1) ──< N equipos_activos (id_cliente):** Un cliente puede poseer varios elevadores; cada activo pertenece obligatoriamente a un cliente registrado.
- **equipos_activos (1) ──< N solicitudes (id_activo):** Un elevador puede generar múltiples solicitudes de mantenimiento preventivo o correctivo a lo largo de su vida útil.
- **clientes (1) ──< N cotizaciones (id_cliente):** Un cliente recibe múltiples cotizaciones comerciales a lo largo del año.
- **equipos_activos (1) ──< N cotizaciones (id_activo):** Cada cotización de refacciones, modernización o póliza se asocia a un elevador específico.
- **solicitudes (1) ── (0..1) cotizaciones (id_solicitud):** Una solicitud técnica puede derivar opcionalmente en una cotización formal elaborada por ventas.
- **cotizaciones (1) ──< N partidas_cotizacion (id_cotizacion):** Una cotización agrupa de 1 a N partidas individuales; exige integridad transaccional mediante ACID.
- **catalogo_conceptos (1) ──< N partidas_cotizacion (id_concepto):** Cada partida referencia un concepto del catálogo maestro, congelando su precio y descripción en la partida.
- **cotizaciones (1) ──< N seguimiento_propuestas (id_cotizacion):** Una cotización puede emitirse y versionarse en distintas propuestas PDF enviadas al cliente durante la negociación.
- **usuarios (1) ──< N bitacora_auditoria (id_usuario):** Cada acción sensible (autenticación, inserción, actualización, bloqueo) queda registrada con el identificador del usuario responsable.

## 10. Prompts específicos para persistencia y base de datos

Siguiendo las pautas metodológicas de la Sección 6 de la guía oficial, el equipo interactuó con el modelo de IA a través de tres prompts estructurados para evaluar la necesidad de base de datos, proponer el diseño mínimo y analizar críticamente los riesgos de un enfoque sin base de datos:

### Prompt 1: Decidir si se requiere base de datos

*A continuación se transcribe el prompt formulado a la IA y las diez respuestas analíticas resultantes:*

> *Prompt usado: 'Actúa como arquitecto de software y diseñador de datos. Con base en nuestro proyecto COTIZA-ELEV, requerimientos (43 RF, 29 RNF), historias de usuario y diseño modular en 9 módulos, analiza si necesitamos base de datos o si basta con archivos u otra forma de almacenamiento. Responde los 10 puntos de la guía sin generar código todavía.'*

| Punto evaluado | Respuesta técnica estructurada (Análisis de Persistencia) |
|---|---|
| 1. ¿El proyecto necesita persistencia? | Sí, absolutamente. Las cotizaciones comerciales tienen un ciclo de negociación que puede durar semanas o meses, los contratos de pólizas duran años y los activos mecánicos requieren trazabilidad de mantenimiento durante décadas. |
| 2. ¿Qué datos deberían guardarse? | Usuarios y credenciales con roles RBAC; directorio de clientes con RFC; fichas técnicas de elevadores; catálogo de tarifas y refacciones con vigencias; solicitudes técnicas de campo; cotizaciones con partidas detalladas; folios y auditoría. |
| 3. ¿Conviene usar base de datos o archivos? | Conviene indiscutiblemente una base de datos relacional. Los archivos planos (JSON/CSV) fallan estrepitosamente ante accesos concurrentes entre técnicos móviles y administradores, carecen de transaccionalidad ACID y exponen datos de precios a técnicos. |
| 4. ¿Qué tipo de base de datos se recomienda? | RDBMS relacional MySQL 8.0 / MariaDB con motor InnoDB, garantizando integridad referencial y transacciones ACID vía PHP PDO. |
| 5. ¿Qué entidades o tablas serían necesarias? | Nueve entidades fundamentales: usuarios, clientes, equipos_activos, catalogo_conceptos, solicitudes, cotizaciones, partidas_cotizacion, seguimiento_propuestas y bitacora_auditoria. |
| 6. ¿Qué atributos tendría cada entidad? | Llaves primarias secuenciales, atributos de negocio tipados estrictamente (DECIMAL para importes, ENUM para estados, VARCHAR con longitud acotada para textos) y marcas de tiempo created_at/updated_at. |
| 7. ¿Qué relaciones existirían? | Relaciones 1 a N de clientes hacia activos y cotizaciones; de usuarios hacia solicitudes y cotizaciones; de cotizaciones hacia partidas y propuestas; y 1 a 0..1 entre solicitudes y cotizaciones. |
| 8. ¿Qué operaciones CRUD serían necesarias? | Creación transaccional multi-partida; consultas con filtros e índices; actualizaciones de estado; y bajas lógicas (soft delete) sin eliminación física de registros transaccionales. |
| 9. ¿Qué módulo manejaría el acceso a datos? | Una clase de infraestructura transversal 'Database' (PDO Singleton) y repositorios especializados dentro de cada módulo funcional. |
| 10. ¿Qué riesgos o complejidades debemos evitar? | Evitar sobreingeniería NoSQL (MongoDB), descartar tipos flotantes (FLOAT/DOUBLE) en dinero sustituyéndolos por DECIMAL(12,2), rechazar el borrado físico en cascada (DELETE CASCADE) y evitar dejar la seguridad de precios en manos del frontend. |

### Prompt 2: Proyectos con base de datos (Diseño mínimo)

> *Prompt usado: 'Actúa como diseñador de base de datos. Nuestro proyecto COTIZA-ELEV requiere almacenar usuarios, clientes, elevadores, tarifas, solicitudes técnicas, cotizaciones multi-partida, folios y bitácora. Ayúdanos a proponer el diseño mínimo de base de datos en 8 puntos concretos según la guía.'*

| Elemento de diseño | Definición técnica adoptada |
|---|---|
| 1. Entidades o tablas: | Las 9 tablas: usuarios, clientes, equipos_activos, catalogo_conceptos, solicitudes, cotizaciones, partidas_cotizacion, seguimiento_propuestas, bitacora_auditoria. |
| 2. Atributos: | Atributos normalizados en 3FN; campos obligatorios y opcionales delimitados para evitar inconsistencias en formularios. |
| 3. Llaves primarias: | Identificadores enteros unsigned AUTO_INCREMENT en todas las tablas (BIGINT en bitacora_auditoria). |
| 4. Relaciones: | Llaves foráneas con restricciones ON DELETE RESTRICT en catálogos y transacciones para proteger la integridad histórica. |

| | |
|---|---|
| 5. Tipos de datos sugeridos: | DECIMAL(12,2) para todos los importes en pesos mexicanos; ENUM para estados finitos; VARCHAR con límites; DATE y TIMESTAMP. |
| 6. Operaciones necesarias: | INSERT con BEGIN TRANSACTION / COMMIT; SELECT con JOIN indexado; UPDATE de transiciones de estado; Soft DELETE mediante bandera de estatus. |
| 7. Módulo responsable de persistencia: | Capa Core (app/Core/Database.php) coordinada con los 9 módulos de aplicación. |
| 8. Requerimientos relacionados: | Trazabilidad completa con los 43 requerimientos funcionales (RF-01 a RF-43). |

## Prompt 3: Análisis crítico de por qué la opción sin base de datos es inviable

*Prompt usado: 'Actúa como arquitecto de software. Analiza críticamente si COTIZA-ELEV podría operar sin base de datos (usando archivos JSON, CSV o almacenamiento en memoria). Responde los 6 cuestionamientos de la guía y explica los riesgos.'*

| Cuestionamiento sobre alternativa sin BD | Dictamen técnico de inviabilidad y riesgos |
|---|---|
| 1. ¿Podemos trabajar sin base de datos? | NO. Un sistema sin base de datos es inviable para el modelo de negocio de ELEVESE. Maneja cotizaciones con validez legal, dinero, cálculo de impuestos, múltiples usuarios simultáneos y activos mecánicos que requieren retención a 5 años (RNF-18). |
| 2. ¿Qué alternativa convendría si se forzara? | Tendría que recurrirse a una combinación de archivos JSON para cotizaciones y CSV para catálogos, pero requeriría programar bloqueos manuales de archivo (flock) propensos a fallas catastróficas. |
| 3. ¿Qué datos serían temporales? | Únicamente los borradores en memoria de sesión activa del navegador antes de presionar el botón 'Guardar Cotización' y tokens CSRF. |
| 4. ¿Qué datos se guardarían en archivos? | Los documentos finales renderizados en PDF (propuestas comerciales en storage/pdf/) y respaldos zipeados para descarga del cliente. |
| 5. ¿Qué módulo manejaría esos archivos? | El módulo M08 (Propuesta PDF y seguimiento) y utilerías de almacenamiento en el Core del sistema. |
| 6. ¿Qué riesgos críticos existen al no usar BD? | 1) Corrupción de cotizaciones por colisión de escrituras simultáneas (race condition).<br>2) Inconsistencia de partidas por falta de rollback atómico.<br>3) Fuga crítica de información: los técnicos podrían abrir el archivo JSON/CSV y conocer las tarifas y márgenes de la empresa.<br>4) Pérdida de integridad referencial: se podrían borrar clientes dejando cotizaciones huérfanas.<br>5) Rendimiento pésimo: parsear archivos JSON gigantescos en cada consulta colapsaría el servidor. |

## 11. Bitácora de uso de IA

En apego a la rúbrica oficial de la práctica, se registraron las seis fases del trabajo asistido por IA, documentando de forma honesta y verificable las sugerencias aceptadas, los errores detectados y las correcciones técnicas aplicadas por el equipo:

| Fase de la práctica | Prompt usado con la IA | Resultado generado por la IA | Qué aceptamos | Qué modificamos (Detección de errores y correcciones) | Responsable |
|---|---|---|---|---|---|
| **1. Decisión de persistencia** | ¿Necesita COTIZA-ELEV base de datos o bastan archivos JSON/CSV para guardar cotizaciones y clientes? | La IA sugirió que para un prototipo rápido bastaba usar archivos JSON locales con un lock de archivos en PHP. | Aceptamos el análisis de entidades necesarias y el volumen estimado de datos. | RECHAZAMOS tajantemente la opción de archivos JSON. Exigimos base de datos relacional MySQL/InnoDB porque el cliente requiere concurrencia multiusuario real y transacciones ACID en partidas. | Orlando Villalobos |
| **2. Diseño de BD o alternativa** | Diseña las tablas necesarias para COTIZA-ELEV con 43 RF, incluyendo usuarios, clientes, activos y cotizaciones. | La IA propuso una colección documental NoSQL en MongoDB o tablas relacionales con tipos FLOAT para precios y llaves foráneas con DELETE CASCADE en clientes y cotizaciones. | Aceptamos la descomposición de 9 entidades y la identificación de llaves primarias. | CORREGIMOS 3 errores graves: 1) Descartamos MongoDB: exigimos MySQL relacional. 2) Cambiamos FLOAT a DECIMAL(12,2) para evitar errores de redondeo en pesos y centavos. 3) Eliminamos DELETE CASCADE y establecimos bajas lógicas (soft delete) con estatus para no destruir históricos. | Orlando Villalobos |
| **3. Plan del módulo** | ¿Qué módulo de datos debemos programar primero: la conexión general, clientes o cotizaciones? | La IA recomendó programar directo el formulario de cotización conectándolo a una tabla plana. | Aceptamos la necesidad de tener datos de prueba semilla (seeders) para probar flujos. | RECHAZAMOS la sugerencia. Diseñamos primero el módulo M01 (Acceso/Seguridad) y la clase central Database (PDO Singleton), ya que sin autenticación RBAC ninguna pantalla puede discriminar entre el técnico y el administrador. | Diego Emilio Alférez |
| **4. Implementación** | Genera el script DDL de creación de tablas en MySQL 8.0 y la clase PHP PDO de conexión segura. | La IA generó el script SQL inicial pero olvidó declarar ENGINE=InnoDB, omitió índices UNIQUE en folios y no incluyó vigencias (vigente_desde/vigente_hasta) en catalogo_conceptos. | Aceptamos la estructura de clases PDO con manejo de excepciones PDOException y prepared statements. | CORREGIMOS el DDL agregando ENGINE=InnoDB en todas las tablas, índices UNIQUE en 'folio_cotizacion', 'rfc', 'numero_serie' y 'correo_electronico', y agregamos vigencias en el catálogo de tarifas. | Orlando Villalobos |
| **5. Revisión** | Revisa si el perfil técnico puede consultar precios mediante inyecciones en la URL o inspectores de navegador. | La IA sugirió ocultar los precios en la vista del técnico usando estilos CSS (display: none) o validación en JavaScript. | Aceptamos el concepto de vistas diferenciadas por rol de usuario. | RECHAZAMOS rotundamente la sugerencia de la IA por ser una vulnerabilidad crítica de seguridad (CWE-602). Establecimos que el backend filtre las columnas a nivel de consulta SQL: el técnico jamás recibe el dato del servidor. | Diego Emilio Alférez |
| **6. Pruebas** | Diseña casos de prueba para validar que una falla en una partida no deje cotizaciones incompletas. | La IA sugirió validar que el usuario no cierre la ventana del navegador mientras guarda. | Aceptamos la formulación de pruebas de estrés y validación de campos obligatorios. | DISEÑAMOS una prueba manual transaccional forzando una excepción en la partida número 3 de una cotización, comprobando que el método rollBack() de PDO canceló toda la operación sin dejar registros huérfanos en la base de datos. | Diego Emilio Alférez |

## 12. Preguntas de cierre

A continuación se responden con precisión técnica y rigor académico las ocho preguntas de cierre estipuladas en la guía oficial de la práctica:

- **1. ¿Qué módulo implementaron primero y por qué?** Implementamos el módulo M01 (Acceso y Seguridad) junto con la clase de infraestructura 'Database' (app/Core/Database.php). Justificación: en COTIZA-ELEV la regla de negocio más importante es que el técnico en campo no debe tener acceso a precios ni catálogos de tarifas. Sin el control de sesiones, autenticación bcrypt y middleware RBAC en funcionamiento, ninguna otra entidad del sistema puede desplegarse de manera segura.
- **2. ¿Qué requerimiento atiende ese módulo?** Atiende los requerimientos funcionales RF-01 (inicio de sesión con correo y contraseña), RF-02 (hash seguro de credenciales), RF-03 (recuperación de contraseña), RF-04 (alta de usuarios y asignación de roles), RF-05 (restricción absoluta de precios al perfil técnico), RF-06 (vistas diferenciadas por rol) y RF-07 (cierre de sesión por inactividad a los 20 minutos).
- **3. ¿El proyecto requiere base de datos o no? ¿Por qué?** Sí, requiere base de datos relacional de manera indispensable. La empresa cotiza elevadores con 5 a 20 partidas por propuesta, lo que exige transacciones ACID atómicas. Asimismo, múltiples técnicos capturan solicitudes en campo mientras dos administradores cotizan en oficina, demandando concurrencia multiusuario con bloqueos de fila, integridad referencial y retención auditada a 5 años.
- **4. ¿Qué módulo será responsable de manejar los datos?** La persistencia la administra la capa central de infraestructura orquestada por la clase 'Database' (PDO Singleton), mientras que cada módulo funcional (M01 a M09) implementa repositorios dedicados (UserRepository, ClientRepository, AssetRepository, QuotationRepository) que aíslan y ejecutan las sentencias SQL preparadas de su respectivo dominio.
- **5. ¿Qué parte del código generó Codex / IA?** La IA generó el borrador inicial de la estructura DDL SQL para la creación de las tablas, la plantilla básica de conexión PDO con manejo de excepciones y las firmas de métodos CRUD para los repositorios de datos.
- **6. ¿Qué revisó o corrigió el equipo?** El equipo corrigió errores de arquitectura graves: sustituyó tipos FLOAT por DECIMAL(12,2) para evitar descuadres en dinero; forzó el motor InnoDB en todas las sentencias CREATE TABLE; eliminó la propuesta de DELETE CASCADE aplicando bajas lógicas (soft deletes); agregó vigencias temporales al catálogo de refacciones; y rechazó la idea de ocultar precios con CSS, obligando a que el backend no envíe datos financieros al técnico.
- **7. ¿Qué prueba manual demuestra que el módulo funciona?** Se ejecutaron tres pruebas manuales verificables:
  a) Migración DDL limpia en MySQL 8.0 creando las 9 tablas con sus índices y llaves foráneas.
  b) Prueba de autenticación con password_verify() verificando que 5 intentos fallidos activan el bloqueo temporal por 15 minutos.
  c) Prueba transaccional multi-partida en la que se simuló la inserción de una cotización con 3 partidas forzando un error en la partida 3; el sistema ejecutó rollBack() exitosamente sin dejar la cotización padre ni partidas huérfanas en la base de datos.
- **8. ¿Qué quedó pendiente para la siguiente semana?** Quedó programado: 1) Conectar el formulario web del M05 (Solicitudes técnicas) con el repositorio SQL; 2) Programar la carga masiva de tarifas desde archivos CSV con validación de filas rechazadas en M04; y 3) Desarrollar el cálculo matemático de subtotales, descuentos e IVA en el Motor de Cálculo (M07) consumiendo directamente las entidades persistidas.

## 13. Tarea para la siguiente semana

De acuerdo con la agenda de continuidad del curso, el equipo definió las siete acciones concretas que se desarrollarán en la siguiente iteración de trabajo:

| Compromiso de la agenda semanal | Acción técnica programada por el equipo COTIZA-ELEV |
|---|---|
| 1. Corregir el primer módulo implementado: | Refactorizar la clase Database para incorporar soporte de transacciones anidadas seguras (savepoints) y mejorar los mensajes de error del módulo M01 (Acceso y Seguridad) para no revelar si un correo existe durante intentos fallidos de autenticación. |
| 2. Actualizar la decisión de persistencia: | Monitorear los tiempos de respuesta de MySQL/InnoDB al ejecutar consultas con JOIN entre cotizaciones, partidas y clientes, confirmando que los índices creados sobre id_cliente e id_cotizacion mantienen la latencia por debajo de 200 ms. |
| 3. Completar el diseño mínimo de base de datos: | Consolidar el archivo de migración reproducible 'database/migrations/01_init_cotiza_elev.sql' y crear un seeder '02_seed_data.sql' con 5 usuarios, 10 clientes, 15 elevadores y 40 conceptos de catálogo para pruebas locales del equipo. |
| 4. Seleccionar el segundo módulo a implementar: | Se seleccionó el módulo M05 (Solicitudes técnicas de campo) junto con M02 (Directorio de Clientes), permitiendo registrar clientes reales y que el perfil técnico capture requerimientos de refacciones desde su vista móvil. |
| 5. Agregar evidencia de commits: | Documentar en el repositorio Git del proyecto los commits atómicos correspondientes a las migraciones SQL, la clase Database y los repositorios iniciales de autenticación con mensajes claros siguiendo Conventional Commits. |
| 6. Actualizar la bitácora de Codex / IA: | Registrar las consultas técnicas de la siguiente semana, enfatizando la interacción para pruebas unitarias de repositorios y la sanitización estricta de entradas contra inyecciones SQL. |
| 7. Preparar dudas técnicas para Codex / IA: | Formular consultas especializadas sobre: a) Manejo de transacciones concurrentes en PHP con nivel de aislamiento READ COMMITTED; b) Buenas prácticas para exportación eficiente de reportes CSV de más de 5,000 registros sin agotar la memoria de PHP. |

## 14. Conclusión y checklist de entrega

El desarrollo de esta práctica de persistencia consolidó los cimientos de datos sobre los cuales operará COTIZA-ELEV. Diseñar las nueve tablas relacionales antes de escribir código evitó errores estructurales graves que habrían costado semanas de retrabajo, tales como el uso de tipos flotantes en cotizaciones financieras, la falta de aislamiento de precios para los técnicos o la pérdida de integridad en cotizaciones multi-partida. La decisión de adoptar MySQL 8.0 con InnoDB y PHP PDO garantiza que la plataforma de ELEVESE sea escalable, segura y completamente desplegable en servidores LAMP convencionales.

### Checklist de verificación de la entrega formal

| Criterio a evaluar | Estado | Evidencia y justificación técnica en el documento |
|---|---|---|
| **Decisión de persistencia documentada** | Realizado | Se fundamentó la adopción de la Ruta B (Base de datos relacional) en la Sección 4 y 5. |
| **Justificación técnica de la decisión** | Realizado | Se analizaron concurrencia, propiedades ACID, seguridad RBAC y despliegue estándar. |
| **Diseño mínimo de base de datos** | Realizado | Se definieron formalmente las 9 tablas relacionales con sus atributos, PK, FK y requerimientos. |
| **Diccionario de datos básico y exhaustivo** | Realizado | Se especificaron 28 campos con tipos de dato, descripción, obligatoriedad y ejemplos reales. |
| **Operaciones de datos delimitadas (CRUD)** | Realizado | Se detallaron las operaciones de Crear, Consultar, Actualizar y Eliminar (Soft Delete) por módulo. |
| **Diagrama ERD visual generado e insertado** | Realizado | Se construyó la Figura 1 en matplotlib con 9 entidades y 10 relaciones perfectamente legibles. |
| **Prompts específicos de persistencia** | Realizado | Se transcribieron y respondieron exhaustivamente los tres prompts de la guía en la Sección 10. |
| **Bitácora completa de uso de IA** | Realizado | Se documentaron las 6 fases obligatorias con aportes aceptados y detección de errores corregidos. |
| **Preguntas de cierre respondidas** | Realizado | Se respondieron con honestidad y detalle técnico las 8 preguntas de cierre en la Sección 12. |
| **Tarea para la siguiente semana** | Realizado | Se establecieron los 7 puntos de la agenda técnica para la siguiente iteración de desarrollo. |

### Firmas de validación y aportación de los integrantes

- **Orlando Villalobos Gutiérrez (Revisor técnico / Diseñador de datos):** Responsable del modelado relacional 3FN, selección del RDBMS MySQL/InnoDB, diseño de transacciones ACID para partidas, definición de la clase Database PDO y supervisión de la coherencia técnica del esquema.
- **Diego Emilio Alférez Vallejo (Tester / Documentador):** Responsable de la matriz de verificación contra los 43 requerimientos funcionales, pruebas manuales de integridad referencial, validación del aislamiento de seguridad de precios para el rol técnico y redacción de la bitácora de IA.